

WEEK 7 PRACTICAL

QUESTION 1

Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Aim

To write a C program that displays the memory addresses of variables belonging to the **code**, **global**, **static**, **heap**, and **stack** segments and understand the Linux process address-space layout.

Program

```
#include <stdio.h>
#include <stdlib.h>

int global_var = 100;

static int static_global_var = 200;

void sample_function() {
    printf("Inside sample function\n");
}

int main() {
    int stack_var = 300;

    static int static_local_var = 400;

    int *heap_var = (int *)malloc(sizeof(int));

    if (heap_var == NULL) {
        perror("malloc");
        return 1;
    }
    *heap_var = 500;
    printf("Memory Addresses:\n\n");

    printf("Code (function)           : %p\n", (void *)sample_function);
    printf("Global variable               : %p\n", (void *)&global_var);
    printf("Static global variable          : %p\n", (void *)&static_global_var);
    printf("Static local variable           : %p\n", (void *)&static_local_var);
    printf("Heap variable                   : %p\n", (void *)heap_var);
    printf("Stack variable                  : %p\n", (void *)&stack_var);

    free(heap_var);

    return 0;
}
```

Output

The exact addresses will be different on every execution because of Linux memory-layout randomization.

Memory Addresses:

```
root@keerthana: ~/OSSP/We  X  +  v
root@keerthana:~# mkdir -p ~/OSSP/Week7
root@keerthana:~# cd ~/OSSP/Week7
root@keerthana:~/OSSP/Week7# pwd
/root/OSSP/Week7
root@keerthana:~/OSSP/Week7# nano memory.c
root@keerthana:~/OSSP/Week7# ls
memory.c
root@keerthana:~/OSSP/Week7# gcc memory.c -o memory
root@keerthana:~/OSSP/Week7# ls
memory memory.c
root@keerthana:~/OSSP/Week7# ./memory
Memory Addresses:
Code / Text address      : 0x57b8b13de1c9
Global variable address  : 0x57b8b13e1010
Static global address    : 0x57b8b13e1014
Static local address     : 0x57b8b13e1018
Heap variable address    : 0x57b8e53092a0
Stack variable address   : 0x7ffeb1ec825c
root@keerthana:~/OSSP/Week7# |
```

Observation

The program displays the addresses of variables located in different parts of the process address space.

Memory Segment	Example	Purpose
Code / Text	sample_function	Contains executable program instructions
Global / Data	global_var	Stores initialized global variables
Static	static_global_var, static_local_var	Static variables have program-long lifetime and are typically stored in the data or BSS segment depending on initialization.
Heap	heap_var	Used for dynamically allocated memory using malloc()

Memory Segment	Example	Purpose
Stack	stack_var	Stores local variables and function-call information

The addresses demonstrate that different types of data are maintained in different regions of the Linux process address space.

Result

The program was successfully implemented to print the addresses of **code, global, static, heap, and stack variables**. The Linux process memory layout was analyzed based on the addresses obtained.

QUESTION 2

Make use of the /proc/maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

Aim

To study the memory organization of a running Linux process using the /proc/<PID>/maps file and memory-analysis commands, and understand virtual memory mappings.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 100;

int main() {
    int local_var = 200;

    printf("Process ID: %d\n", getpid());
    printf("Global variable address: %p\n", (void *)&global_var);
    printf("Stack variable address : %p\n", (void *)&local_var);

    printf("\nProcess is running...\n");
    printf("Open another terminal and execute:\n");
    printf("cat /proc/%d/maps\n", getpid());

    printf("\nPress Enter to terminate...\n");
    getchar();

    return 0;
}
```

Output

```
root@keerthana: ~/OSSP/We  X + v
root@keerthana:~# cd ~/OSSP/Week7
root@keerthana:~/OSSP/Week7# cat /proc/2468/maps
cat: /proc/2468/maps: No such file or directory
root@keerthana:~/OSSP/Week7# ./process
Process ID: 456
Process is running...
./process
^C
root@keerthana:~/OSSP/Week7# |
```

Useful Memory Analysis Commands

To examine the process mappings:

```
cat /proc/1722/maps
```

Sample /proc/1722/maps Output

```
root@keerthana: ~/OSSP/We  X + v
root@keerthana:~# cd ~/OSSP/Week7
root@keerthana:~/OSSP/Week7# cat /proc/456/maps
653ff067e000-653ff067f000 r--p 00000000 08:30 45964 /root/OSSP/Week7/process
653ff067f000-653ff0680000 r-xp 00001000 08:30 45964 /root/OSSP/Week7/process
653ff0680000-653ff0681000 r--p 00002000 08:30 45964 /root/OSSP/Week7/process
653ff0681000-653ff0682000 r--p 00002000 08:30 45964 /root/OSSP/Week7/process
653ff0682000-653ff0683000 rw-p 00003000 08:30 45964 /root/OSSP/Week7/process
65401ee72000-65401ee93000 rw-p 00000000 00:00 0 [heap]
7ff061c00000-7ff061c28000 r--p 00000000 08:30 49839 /usr/lib/x86_64-linux-gnu/libc.so.6
7ff061c28000-7ff061db0000 r-xp 00028000 08:30 49839 /usr/lib/x86_64-linux-gnu/libc.so.6
7ff061db0000-7ff061dff000 r--p 001b0000 08:30 49839 /usr/lib/x86_64-linux-gnu/libc.so.6
7ff061dff000-7ff061e03000 r--p 001fe000 08:30 49839 /usr/lib/x86_64-linux-gnu/libc.so.6
7ff061e03000-7ff061e05000 rw-p 00202000 08:30 49839 /usr/lib/x86_64-linux-gnu/libc.so.6
7ff061e05000-7ff061e12000 rw-p 00000000 00:00 0
7ff061fb5000-7ff061fb8000 rw-p 00000000 00:00 0
7ff061fb8000-7ff061fbf000 rw-p 00000000 00:00 0
7ff061fbf000-7ff061fc3000 r--p 00000000 00:00 0 [vvar]
7ff061fc3000-7ff061fc5000 r--p 00000000 00:00 0 [vvar_vclock]
7ff061fc5000-7ff061fc7000 r-xp 00000000 00:00 0 [vdso]
7ff061fc7000-7ff061fc8000 r--p 00000000 08:30 49836 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ff061fc8000-7ff061ffc000 r-xp 00001000 08:30 49836 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ff061ffc000-7ff061ffd000 r--p 0002c000 08:30 49836 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ff061ffd000-7ff061fff000 r--p 00036000 08:30 49836 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ff061fff000-7ff062001000 rw-p 00038000 08:30 49836 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ffc7ea7b000-7ffc7ea9c000 rw-p 00000000 00:00 0 [stack]
root@keerthana:~/OSSP/Week7# |
```

To view memory statistics:

cat /proc/1722/status

```
root@keerthana: ~/OSSP/We  ×  +  ∨
root@keerthana:~/OSSP/Week7# cat /proc/456/status
Name:    process
Umask:   0022
State:   S (sleeping)
Tgid:    456
Ngid:    0
Pid:     456
PPid:    438
TracerPid: 0
Uid:     0      0      0      0
Gid:     0      0      0      0
FDSize:  256
Groups:  0
NSTgid:  456
NSpid:   456
NSpgid:  456
NSSid:   438
Kthread: 0
VmPeak:  2688 kB
VmSize:  2688 kB
VmLck:    0 kB
VmPin:    0 kB
VmHWM:   1668 kB
VmRSS:   1668 kB
RssAnon:          96 kB
RssFile:         1572 kB
RssShmem:         0 kB
VmData:   224 kB
VmStk:    132 kB
VmExe:     4 kB
VmLib:   1748 kB
VmPTE:     48 kB
```

To view detailed memory information:

cat /proc/1722/smmaps

```
root@keerthana: ~/OSSP/Week7# cat /proc/456/smmaps
653ff067e000-653ff067f000 r--p 00000000 08:30 45964
    /root/OSSP/Week7/process
Size:                               4 kB
KernelPageSize:                     4 kB
MMUPageSize:                         4 kB
Rss:                                 4 kB
Pss:                                 2 kB
Pss_Dirty:                           0 kB
Shared_Clean:                        4 kB
Shared_Dirty:                         0 kB
Private_Clean:                       0 kB
Private_Dirty:                       0 kB
Referenced:                          4 kB
Anonymous:                           0 kB
KSM:                                 0 kB
LazyFree:                            0 kB
AnonHugePages:                       0 kB
ShmemPmdMapped:                      0 kB
FilePmdMapped:                       0 kB
Shared_Hugetlb:                      0 kB
Private_Hugetlb:                     0 kB
Swap:                                0 kB
SwapPss:                             0 kB
Locked:                              0 kB
THPeligible:                          0
VmFlags: rd mr mw me sd
653ff067f000-653ff0680000 r-xp 00001000 08:30 45964
    /root/OSSP/Week7/process
```

You can also use:

pmap 1722

```
root@keerthana:~/OSSP/Week7# pmap 456
456:  ./process
0000653ff067e000      4K r---- process
0000653ff067f000      4K r-x-- process
0000653ff0680000      4K r---- process
0000653ff0681000      4K r---- process
0000653ff0682000      4K rw--- process
000065401ee72000    132K rw--- [ anon ]
00007ff061c00000    160K r---- libc.so.6
00007ff061c28000   1568K r-x-- libc.so.6
00007ff061db0000    316K r---- libc.so.6
00007ff061dff000     16K r---- libc.so.6
00007ff061e03000      8K rw--- libc.so.6
00007ff061e05000     52K rw--- [ anon ]
00007ff061fb5000     12K rw--- [ anon ]
00007ff061fbd000      8K rw--- [ anon ]
00007ff061fbf000     16K r---- [ anon ]
00007ff061fc3000      8K r---- [ anon ]
00007ff061fc5000      8K r-x-- [ anon ]
00007ff061fc7000      4K r---- ld-linux-x86-64.so.2
00007ff061fc8000    172K r-x-- ld-linux-x86-64.so.2
00007ff061ff3000     40K r---- ld-linux-x86-64.so.2
00007ff061ffd000      8K r---- ld-linux-x86-64.so.2
00007ff061fff000      8K rw--- ld-linux-x86-64.so.2
00007ffc7ea7b000    132K rw--- [ stack ]
  total                2688K
root@keerthana:~/OSSP/Week7# |
```

Observation

The `/proc/<PID>/maps` file shows the **virtual memory regions** assigned to a running process.

Common mappings include:

- **Executable/text region** — contains the program's machine instructions.
- **Read-only region** — contains read-only data and other protected mappings.
- **Data region** — contains initialized and writable program data.
- **Heap** — used for dynamically allocated memory.
- **Shared libraries** — libraries such as the C library are mapped into the process's virtual address space.
- **Stack** — contains local variables and function-call information.
- **VDSO/VVAR** — special kernel-provided mappings used for certain system operations and timing-related functionality.

The permissions shown in `/proc/<PID>/maps` describe what operations are allowed:

r = read

w = write

x = execute

p = private mapping

s = shared mapping

Result

The memory organization of a running Linux process was successfully studied using `/proc/<PID>/maps`, `/proc/<PID>/status`, `/proc/<PID>/smaps`, and `pmap`. The experiment demonstrated how **virtual memory mappings** divide a process's address space into code, data, heap, stack, libraries, and other regions.